

As your senior dev, I'm telling you right now: text-to-SQL is a minefield of hallucinated columns, dropped WHERE clauses, and security nightmares. If we don't lock this down with strict state management and database constraints, the AI *will* drop a table or leak corporate data.

Here is the ruthless, step-by-step implementation plan to build exactly what is outlined in your architecture document.

Phase 1: Python Foundation & State Machine (Days 1-2)

We isolate the AI logic in its own Python service. Spring Boot will just be a proxy.

1. **Stand up FastAPI:** Create a lightweight Python FastAPI service that exposes a single POST endpoint for Spring Boot to hit.
2. **Define the State (TypedDict):** This is non-negotiable. Define your LangGraph state with strict typing to prevent the agents from passing garbage to each other. Your state must include: question, sql_query, query_result, error, final_answer, visualization_code, is_in_scope, and iteration_count.
3. **Wire the Graph:** Set up the LangGraph nodes and edges. The flow must be strictly enforced: Guardrails -> SQL Agent -> Execute -> (If Error -> Error Agent -> Execute) -> Analysis Agent -> Decide Graph Need -> (If Needed -> Visualization Agent). Set a hard cap of 3 retries on the execution loop to prevent infinite LLM billing cycles.

Phase 2: Agent Implementation (Days 3-4)

Keep your system prompts tight. Do not let the agents bleed into each other's responsibilities. Use gpt-4o-mini (or Claude/Ollama) as your LLM.

1. **Guardrails Agent:** Build this to strictly identify greetings (respond and terminate) or out-of-scope questions (reject and terminate). It must only allow e-commerce data analysis queries.
2. **SQL Agent:** Provide it the database schema. Its only job is to generate raw, valid SQL. Tell it: "Generate only valid SQL queries without any formatting or explanation".
3. **Error Agent:** If the SQL fails, pass the raw database error to this agent along with the schema so it can fix syntax or logic errors.
4. **Analysis & Viz Agents:** The Analysis Agent translates the raw SQL results into natural language insights. If a chart is needed, the Visualization Agent generates *only* executable Plotly code.

Phase 3: The Security Sandbox (Day 5 - Critical)

If you skip this, you will leak data or get your database wiped.

1. **Read-Only Database Credentials:** The Python service must execute the LLM's SQL queries using a strictly locked-down, read-only database user. No INSERT, UPDATE, or DELETE privileges.
2. **Role-Based Scope Injection:** The Spring Boot backend must pass the user's role and ID to Python.
 - **Individual:** Must inject a hard WHERE user_id = X into their context.
 - **Corporate:** Must inject a hard WHERE store_id = Y into their context.
 - **Admin:** Gets full platform access.

Phase 4: Full-Stack Integration (Days 6-7)

Wire the two backends and update the UI.

1. **Spring Boot Proxy:** Implement the /api/chat/ask endpoint in Spring Boot to accept the Angular request, append the user's secure context (Role/ID), and forward it to the Python FastAPI service.
2. **Handle Plotly in Angular:** Update your existing ChatWidgetComponent. If the backend response includes visualization_code, use a frontend Plotly wrapper (like angular-plotly.js) to evaluate and render the chart inside the chat bubble.